



Movie Watchlist

Bulut Mimarilerinde Test Mühendisliği — Dönem Projesi

FastAPI · Docker · Kubernetes · GitHub Actions · Playwright · Testcontainers

 Çağdaş Karataş

 Eren Şeremet

 Haziran 2026

 MTH2526-B25

← Geri

İleri →

Problem & Çözüm

Problem

- İzlenecek filmler not defterinde kaybolup gidiyor
- İzlenip izlenmediği unutuluyor, tekrar izlenebiliyor
- Puan verilemiyor, istatistik görülemiyor
- Her cihazdan erişim yok, merkezi veri yok

Çözüm — Movie Watchlist API

- Film ekle, listele, ara, sil — 8 REST endpoint
- İzlendi işaretle, tarih otomatik kaydedilir
- 1-10 arası puan ver, ortalama istatistik al
- Web arayüzü (Streamlit) + interaktif Swagger UI

Teknoloji Seçimleri ve Neden?

- **FastAPI** — modern, async, otomatik /docs üretir
- **SQLAlchemy ORM** — SQL yazmadan Python ile DB işlemleri
- **Pydantic v2** — gelen veri otomatik doğrulanır, 422 döner
- **Streamlit** — sadece Python ile web UI, E2E test yüzeyi
- **SQLite** lokal dev / **PostgreSQL** production & test

Mimari Karar: Servis Katmanı

- Endpoint'ler sadece HTTP katmanı — iş mantığı yok
- `movie_service.py` tüm DB işlemlerini yapıyor
- Servis katmanı sayesinde endpoint'ten bağımsız test edilebiliyor
- HTTPException sadece endpoint'te — service `None` döner

API Endpoint'leri

Method	Path	Açıklama	Response
GET	/health	Servis sağlık kontrolü	200
POST	/movies	Yeni film ekle	201
GET	/movies	Filmli listele (watched filtresi)	200
GET	/movies/search	Başlık veya yönetmen ara	200
GET	/movies/{id}	Tek film getir	200/404
PATCH	/movies/{id}	Film güncelle (kısmi)	200/404
DELETE	/movies/{id}	Film sil	204/404
GET	/stats	İstatistikler	200

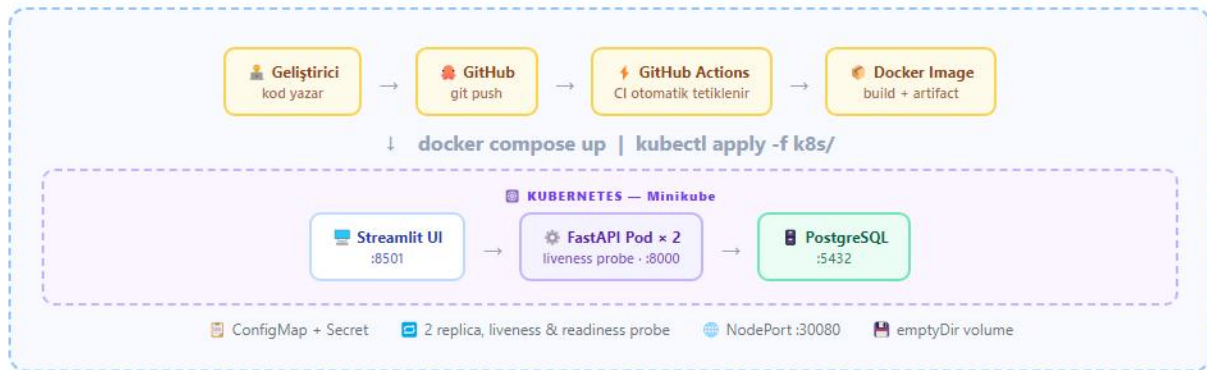
Movie Alanları

- `title` — zorunlu, max 200 karakter
- `director`, `year`, `genre`
- `watched` — default False
- `rating` — 1-10, validator ile
- `added_at`, `watched_at` — otomatik

Önemli Detaylar

- `/search`, `/id` 'dan önce tanımlı
- PATCH → `exclude_unset=True`
- `watched=True` → `watched_at` otomatik set
- CORS middleware → Streamlit erişimi

Mimari



Docker Compose (Lokal)

- 3 servis: postgres → app → ui (sıralı bağımlılık)
- postgres healthcheck geçmeden app başlamaz
- `DATABASE_URL` env var ile bağlanır

Kubernetes (Minikube)

- `imagePullPolicy: Never` → local image cache
- Pod çökerse Kubernetes otomatik yeniden başlatır
- 2 replica → yük dağılımı + yüksek erişilebilirlik

Docker — Multi-Stage Build

⚙️ Aşama 1 — Builder

- `python:3.11-slim` base image
- `gcc`, `libpq-dev` — derleme araçları
- `pip install -r requirements.txt`
- Kütüphaneler `~/local` 'e kurulur

📦 Neden Multi-Stage?

- `gcc` production'da gereksiz — ama derleme için şart
- Tek aşamalı olsaydı: ~400 MB
- Multi-stage sonucu: ~**73 MB** (%80 küçülme)
- Küçük image = hızlı deploy + az saldırı yüzeyi

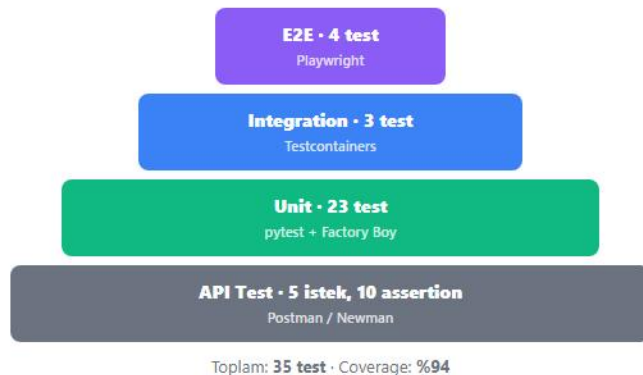
🚀 Aşama 2 — Runtime

- Sadece `libpq5` ve `curl` — derleme araçları yok
- Builder'dan `COPY --from=builder /root/.local`
- Sadece `src/` klasörü kopyalanır
- Healthcheck: `curl /health` her 30 saniye

🐳 docker-compose.yml

- 3 servis: `postgres`, `app`, `ui`
- `depends_on: condition: service_healthy`
- Postgres hazır olmadan app başlamaz
- `pg_data` named volume — veri kalıcı

Test Stratejisi — Test Piramidi



● Unit (23 test) — In-Memory SQLite

- `StaticPool` → tüm bağlantılar aynı DB'yi görür
- `MovieFactory` ile Faker — sahte veri üretimi
- 14 service testi + 9 endpoint testi
- Her test sıfırdan DB açar, biter bitmez siler

● Integration (3 test) — Gerçek PostgreSQL

- Testcontainers → Docker'dan `postgres:16` çeker
- `scope="session"` → tek container, 3 test koşar
- Her test öncesi `TRUNCATE TABLE` ile temizlik
- "ORM sorguları PostgreSQL'de çalışıyor mu?" doğrulanır

● E2E (4 test) — Playwright Headless Chromium

- Film ekle → listede görünüyor mu?
- İzlendi işaretle → UI'da ☒ çıkıyor mu?
- Arama → sadece eşleşen film görünüyor mu?
- Sil → listeden kayboluyor mu?

CI/CD Pipeline — GitHub Actions



Tetikleyiciler

- Her `push` → main branch
- Her `pull_request` → main branch
- Adım başarısız → sonraki job çalışmaz (`needs:`)
- newman + k8s-validate paralel çalışır

k8s-validate — Kubeconform

- `kubectl --dry-run` cluster bağlantısı arıyor — CI'da yok
- `kubeconform` offline çalışır, JSON schema kullanır
- ConfigMap, Secret, Deployment, Service doğrulanır
- Hatalı YAML veya geçersiz alan varsa başarısız olur

Newman Smoke Test

- docker-build artifact'ten image yüklenir
- Container başlatılır, `/health` hazır olana kadar beklenir
- `newman run collection.json` → 5 istek, 10 assertion
- Test biter → container silinir (`if: always()`)

Coverage Artifact

- `coverage.xml` Actions'a upload edilir
- İstenen minimum: `--cov-fail-under=70`
- Gerçekleşen: **%94**
- Hangi satırlar eksik → `--cov-report=term-missing`

Sayılar & Sonuçlar

35

Toplam Test

%94

Code Coverage

**~73
MB**

Docker Image

2

K8s Replica

6

CI Job

8

REST Endpoint

Test Dağılımı

- 23 unit · 3 integration · 4 E2E
- 5 Postman isteği, 10 assertion — 0 hata
- Hedef coverage: %70 → Gerçekleşen: **%94**
- src/main.py → %93 · movie_service.py → %98

Docker & K8s

- Multi-stage build: tek aşamalı ~400MB → **73MB**
- 2 replica · liveness probe her 10 sn
- Startup: `init_db()` tabloları oluşturur
- NodePort :30080 → dışarıdan erişim

Pipeline

- 6 job: lint → test → integration → build → newman + k8s
- Her push'ta otomatik çalışır
- Newman: 5 HTTP isteği, 0 başarısız
- Tüm job'lar yeşil 

Öğrendiklerim & Zorluklar

⚡ Streamlit + Playwright (Headless)

Streamlit'in WebSocket tabanlı buton mekanizması headless Chromium'da API çağrısı tetiklemedi. Çözüm: yazma işlemleri direkt API üzerinden yapıldı; UI render'ı Playwright ile doğrulandı.

🗄 In-Memory SQLite "no such table" Hatası

SQLAlchemy'de her bağlantı ayrı in-memory DB açıyor. Engine tabloları oluştururken TestClient farklı bağlantıdan sorguluyordu. Çözüm: `StaticPool` ile tüm bağlantılar aynı DB'yi paylaştı.

🔌 CI'da K8s Manifest Validasyonu

`kubectl --dry-run=client` bile cluster bağlantısı arıyordu (OpenAPI şeması için). Çözüm: `kubeconform` ile offline JSON schema doğrulaması yapıldı.

✦ En Değerli Öğrenimler

- Test piramidi: altta ucuz/hızlı, üstte gerçekçi — her katmanın amacı farklı
- Multi-stage Docker: derleme araçları production'da gereksiz
- Testcontainers: gerçek DB, sıfır kurulum — mock'a gerek yok
- Servis katmanı: test edilebilirliği doğrudan etkiliyor
- CI pipeline: her push'ta güvende hissettiriyor

📦 İleride Eklenebilecekler

- Prometheus + Grafana: latency, error rate, throughput
- k6 ile performans testi: p95 latency ölçümü
- LocalStack S3: film poster yükleme
- GHCR'a image push adımı (CD tamamlanır)